

Vediamo ora l'algebra alla base dei moderni calcolatori : l'**algebra booleana** : ovvero quell'algebra definita da George Boole che serviva per verificare la veridicità o la falsità di alcune espressioni in linguaggio naturali. Questa algebra poi è stata ripresa ed aumentata da Shannon , il quale la usa per studiare e progettare circuiti con relè . In dettaglio una funzione booleana si caratterizza così:

$$\boxed{S=\{0,1\} \quad \begin{array}{l} f: \{0,1\} \mapsto \{0,1\} \\ y = f(x) \end{array} \quad \begin{array}{l} f: \{0,1\}^n \mapsto \{0,1\} \\ y = f(x_1, x_2, \dots, x_n) \end{array}}$$

Dove la prima colonna mostra il i valori che può assumere la variabile, la seconda il dominio della variabile, mentre l'ultima mostra il caso di multivariabile. Vediamo le proprietà:

Prodotto logico	*
Somma logica	+
Assiomi fondamentali	$\forall a, b \in S; a + b \in S; a \cdot b \in S$ (chiusura) $\exists 0 \in S \forall a \in S, a + 0 = a; \exists 1 \in S \forall a \in S, a \cdot 1 = a$ (elemento identità) $a + b = b + a$ (proprietà commutativa) $(a + b) + c = a + (b + c); (a \cdot b) \cdot c = a \cdot (b \cdot c)$ (proprietà associativa) $a + b \cdot c = (a + b) \cdot (a + c); a \cdot (b + c) = a \cdot b + a \cdot c$ (proprietà distributiva) $\forall a \in S \exists \bar{a} \in S a + \bar{a} = 1; a \cdot \bar{a} = 0$ (elemento inverso) $ S = 2^n; n = 1, 2, 3, \dots$ (cardinalità)
Legge dualità	Ottenuta scambiando + con * e 0 con 1
Idempotenza	$A + A = A$ e $A \cdot A = A$
Annichilatori funzionali	$A + 1 = 1$ e $A \cdot 0 = 0$
Assorbimento	$A + A \cdot b = A$ e $A \cdot (A + b) = A$
Teorema di De Morgan (permette la sostituzione di + e * in funzione degli altri due operatori)	$\overline{a + b} = \bar{a} \cdot \bar{b} \quad \overline{a \cdot b} = \bar{a} + \bar{b}$

Andiamo a vedere ora come rappresentare le funzioni di commutazione : principalmente 3 possibili modi :

1. Forma tabellare (tabelle di verità):
 - a. Ovvero espresse come una relazione tra le variabili di input e quelle di output : x= (x1,x2,...,xn)

x	x ₁	x ₂	x ₃	y
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	0

- b.
- c. In generale ogni funzione può essere espressa come :

d.

Negazione	
x_1	y
0	1
1	0

Somma logica		
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Prodotto logico		
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

- d.
- e. Vediamo ora quelli derivati : X-OR, NAND,NOR, X-NOR:

X-OR (somma modulo 2) $a \oplus b = \bar{a}b + a\bar{b}$	<table> <tr><th>x₁</th><th>x₂</th><th>y</th></tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x ₁	x ₂	y	0	0	0	0	1	1	1	0	1	1	1	0	$a \oplus b = b \oplus a$ (proprietà commutativa); $a \oplus (b \oplus c) = (a \oplus b) \oplus c$ (proprietà associativa); $a \oplus a = 0$; $a \oplus \bar{a} = 1$; $a \oplus 1 = \bar{a}$; $\bar{a} \oplus b = a \oplus \bar{b} = \overline{a \oplus b}$
x ₁	x ₂	y															
0	0	0															
0	1	1															
1	0	1															
1	1	0															
NAND (not and) $a \downarrow b = \overline{a \cdot b} = \bar{a} + \bar{b}$	<table> <tr><th>x₁</th><th>x₂</th><th>y</th></tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	x ₁	x ₂	y	0	0	1	0	1	1	1	0	1	1	1	0	$a \downarrow b = b \downarrow a$ (proprietà commutativa); $a \downarrow 1 = \bar{a}$; $a \downarrow 0 = 1$; $a \downarrow \bar{a} = 1$; $a \downarrow (b \downarrow c) \neq (a \downarrow b) \downarrow c$ (l'operatore <u>non</u> è associativo)
x ₁	x ₂	y															
0	0	1															
0	1	1															
1	0	1															
1	1	0															

NOR (duale di nand) : negazione dell'or $a \downarrow b = \overline{a + b} = \bar{a} \cdot \bar{b}$	x_1	x_2	y	$a \downarrow b = b \downarrow a$ (proprietà commutativa); $a \downarrow 1 = 0$; $a \downarrow 0 = \bar{a}$; $a \downarrow \bar{a} = 0$; $a \downarrow (b \downarrow c) \neq (a \downarrow b) \downarrow c$ (l'operatore <u>non</u> è associativo)
	0	0	1	
	0	1	0	
	1	0	0	
	1	1	0	
X-NOR (somma modulo 2) negazione dell'X-OR	x_1	x_2	y	$a \odot b = b \odot a$ (proprietà commutativa); $a \odot (b \odot c) = (a \odot b) \odot c$ (proprietà associativa); $a \odot 1 = a$; $a \odot \bar{a} = 0$; $a \odot 0 = \bar{a}$;
	0	0	1	
	0	1	0	
	1	0	0	
	1	1	1	

Vediamo ora in dettaglio il **teorema di shannon** : una qualunque funzione può essere scritta così :
 $y=f(x_1, x_2, \dots, x_n)$ la quale può essere scritta in forma duale :

$$f(x_1, x_2, \dots, x_n) = x_1 \cdot f(1, x_2, \dots, x_n) + \bar{x}_1 \cdot f(0, x_2, \dots, x_n)$$

$$f(x_1, x_2, \dots, x_n) = (x_1 + f(0, x_2, \dots, x_n)) \cdot (\bar{x}_1 + f(1, x_2, \dots, x_n))$$

Dove i termini che moltiplicano/ sommano ad x_1 sono chiamati residui . Questa cosa è vera nel caso di variabili indipendenti . In generale questo teorema può essere applicato iterativamente a tutte le variabili della funzione :

$$f(x_1, \dots, x_n) = \bar{x}_1 \cdot \bar{x}_2 \cdot \dots \cdot \bar{x}_n f(0, 0, \dots, 0) + x_1 \cdot \bar{x}_2 \cdot \dots \cdot \bar{x}_n f(1, 0, \dots, 0) +$$

$$+ \bar{x}_1 \cdot x_2 \cdot \dots \cdot \bar{x}_n f(0, 1, \dots, 0) + \dots +$$

$$+ x_1 \cdot x_2 \cdot \dots \cdot \bar{x}_n f(1, 1, \dots, 0) + x_1 \cdot x_2 \cdot \dots \cdot x_n f(1, 1, \dots, 1)$$

Generalizzando quanto detto finora :

$$x_1^{\alpha_1} x_2^{\alpha_2} \dots x_n^{\alpha_n} f(\alpha_1, \alpha_2, \dots, \alpha_n)$$

con $\alpha_i \in \{0,1\}$ e $x_i^{\alpha_i} = x_i$ se $\alpha_i = 1$, $x_i^{\alpha_i} = \bar{x}_i$ se $\alpha_i = 0$

In generale ogni funzione si può scrivere in due modi : **somme di prodotti (SOP)** oppure nella dualità **prodotti di somme (POS)** . In dettaglio :

- Somma di prodotti (SOP): si prendono gli uno della funzione e si negano gli zero .

$$a. \quad f(x_1, x_2, \dots, x_n) = \sum_{k=0}^{2^n-1} m_k f(k)$$

- Il termine m_k viene chiamato *mintermine* ed è nella forma $x_1^{\alpha_1} x_2^{\alpha_2} \dots x_n^{\alpha_n}$
- Da notare che in questa espressione le variabili compaiono 1 ed una sola volta in forma affermata o negata.

- Dualmente alla somma di prodotti ed usando De Morgan , arriviamo al Prodotto di somme (POS) : si affermano gli zero e si negano gli uno :

$$\overline{f(x_1, x_2, \dots, x_n)} = \sum_{k=0}^{2^n-1} m_k \overline{f(k)}$$

- $$f(x_1, x_2, \dots, x_n) = \overline{\sum_{k=0}^{2^n-1} m_k \overline{f(k)}} = \prod_{k=0}^{2^n-1} \overline{m_k \overline{f(k)}} = \prod_{k=0}^{2^n-1} (\overline{m_k} f(k))$$
- Dove Il termine $\overline{m_k}$ viene chiamato *maxtermine* ed è nella forma $\overline{m_k} = \prod_{i=0}^{n-1} x_i^{\alpha_i}$, con $\alpha_i \in \{0,1\}$ e $x_i^{\alpha_i} = x_i$ se $\alpha_i = 0$, $x_i^{\alpha_i} = \bar{x}_i$ se $\alpha_i = 1$

Vediamo un esempio :

k	x_1	x_2	x_3	$f(k)$
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

Dove per mintermini si prendono le somme di $K=0,4,5,7$, mentre per maxtermini si prende prodotto di $K=1,2,3,6$.

Per maxtermini :

$$f(x_1, x_2, x_3) = (x_1 + x_2 + \bar{x}_3)(x_1 + \bar{x}_2 + x_3)(x_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_1 + \bar{x}_2 + x_3)$$

Per mintermini :

$$f(x_1, x_2, x_3) = \bar{x}_1 \bar{x}_2 \bar{x}_3 + x_1 \bar{x}_2 \bar{x}_3 + x_1 \bar{x}_2 x_3 + x_1 x_2 x_3$$

Vediamo ora 3 forme : Canoniche , Decimale , Semplificate.

1. **Canoniche** : rappresentano una funzione di commutazione : si usa un truccetto , ovvero se variabile non compare si moltiplica per affermato +negato:

a.

$$x_1x_3 + \overline{x_1}(x_2 + \overline{x_3}) = x_1x_3(x_2 + \overline{x_2}) + \overline{x_1}x_2(x_3 + \overline{x_3}) + \overline{x_1}\overline{x_3}(x_2 + \overline{x_2})$$

b.

$$= x_1x_2x_3 + x_1\overline{x_2}x_3 + \overline{x_1}x_2x_3 + \overline{x_1}x_2\overline{x_3} + \overline{x_1}\overline{x_2}x_3 + \overline{x_1}\overline{x_2}\overline{x_3}$$

2. **Decimale** : si rappresenta la funzione come prodotti (max) o somme (min):

a.

$$f(k) = \sum (0,4,5,7) = \prod (1,2,3,6)$$

b.

k	x_1	x_2	x_3	$f(k)$
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

3. Per le ultime invece si ha che : dato che non si è certi che un metodo prevalga su altro, ci si affida all'esperienza , quindi si cerca di ridurre al minimo la funzione attraverso metodi analitici e/o algoritmici .

Vediamo un esempio :

Semplificazione analitica: un esempio

- Consideriamo la funzione:

$$f(x, y, z, w) = xyzw + xyz\overline{w} + xy\overline{z}w + xy\overline{z}\overline{w} + \overline{x}yzw$$
- Per la proprietà dell'idempotenza, un termine può essere ripetuto più di una volta senza modificare il risultato
- Primo e secondo mintermine: $xyzw + xyz\overline{w} = xyz$
- Primo e quarto mintermine: $xyzw + xy\overline{z}w = xyw$
- Primo e quinto mintermine: $xyzw + \overline{x}yzw = yzw$
- Secondo e terzo mintermine: $xyz\overline{w} + xy\overline{z}\overline{w} = xy\overline{w}$
- Otteniamo:

$$f(x, y, z, w) = xyz + xyw + yzw + xy\overline{w}$$
- Proseguendo:

$$f(x, y, z, w) = xyz + xyw + yzw + xy\overline{w}$$
- Secondo e ultimo termine: $xyw + xy\overline{w} = xy$

$$f(x, y, z, w) = xy + xyz + yzw$$
- Per la legge dell'assorbimento ($a + a \cdot b = a$):

$$f(x, y, z, w) = xy + yzw$$
- In sintesi, abbiamo trovato la seguente uguaglianza:

$$f(x, y, z, w) = xyzw + xyz\overline{w} + xy\overline{z}w + xy\overline{z}\overline{w} + \overline{x}yzw = xy + yzw$$

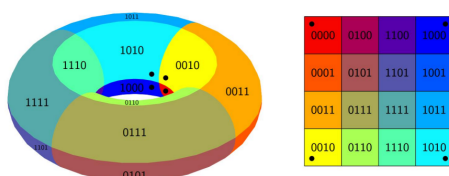
Andiamo a vedere ora in dettaglio il funzionamento delle mappe Karnaugh (mappe k): innanzitutto diciamo che è un'ulteriore rappresentazione delle tabelle di verità , le quali sono organizzate in rettangoli o quadrati a seconda del numero dei variabili che compaiono nella tabella. Da notare che le celle della tabella contigue sono caratterizzate da Codice di Gray , quindi da distanza di Humming pari ad 1 : tra celle contigue cambia al massimo 1 singolo bit .

Se tra due celle contigue il valore di una variabile cambia, viene ignorata in base alla legge della somma tra una variabile ed il suo negato che vale 1 . Con questo metodo è possibile semplificare funzioni fino a 6 variabili . Andiamo a vedere in generale :

$\begin{array}{c cc} & 0 & 1 \\ \hline 0 & f(00) & f(01) \\ \hline 1 & f(10) & f(11) \end{array}$	$\begin{array}{c cccc} & 00 & 01 & 11 & 10 \\ \hline 0 & f(000) & f(010) & f(110) & f(100) \\ \hline 1 & f(001) & f(011) & f(111) & f(101) \end{array}$	$\begin{array}{c cccc} & 00 & 01 & 11 & 10 \\ \hline 00 & f(0000) & f(0100) & f(1100) & f(1000) \\ \hline 01 & f(0001) & f(0101) & f(1101) & f(1001) \\ \hline 11 & f(0011) & f(0111) & f(1111) & f(1011) \\ \hline 10 & f(0010) & f(0110) & f(1110) & f(1010) \end{array}$
2 variabili	3 variabili	4 variabili

Ricordiamo della distanza di humming pari ad 1 .

Piccola osservazione : nel caso di tabella k a 4 variabili (16 combinazioni) , la si può pensare come uno sviluppo toroidale :



Nel caso invece di funzione a 5 variabili : si usano 2 tabelle 4*4 , ma con una particolarità : nella prima tabella si considera la quinta variabile = 0 , mentre nella seconda tabella la si considera = 1 : in questo caso l'adiacenza si vede anche facendo la sovrapposizione delle tabelle :

x_4x_3 \ x_2x_1	00	01	11	10
00	$f(00000)$	$f(01000)$	$f(11000)$	$f(10000)$
01	$f(00010)$	$f(01010)$	$f(11010)$	$f(10010)$
11	$f(00110)$	$f(01110)$	$f(11110)$	$f(10110)$
10	$f(00100)$	$f(01100)$	$f(11100)$	$f(10100)$

$x_4 = 0$

x_4x_3 \ x_2x_1	00	01	11	10
00	$f(00001)$	$f(01001)$	$f(11001)$	$f(10001)$
01	$f(00011)$	$f(01011)$	$f(11011)$	$f(10011)$
11	$f(00111)$	$f(01111)$	$f(11111)$	$f(10111)$
10	$f(00101)$	$f(01101)$	$f(11101)$	$f(10101)$

$x_4 = 1$

Nel caso di funzione a 6 variabili , il procedimento è analogo : solo che le tabelle ora diventano 4 in più : per ognuna delle due variabili in più, si usano le opportune configurazioni (00,01,11,10) ed oltre alla sovrapposizione delle tabelle , si usa anche la sovrapposizione ad "L" :

x_5x_4 \ x_3x_2	00	01	11	10
00	$f(000000)$	$f(010000)$	$f(110000)$	$f(100000)$
01	$f(000100)$	$f(010100)$	$f(110100)$	$f(100100)$
11	$f(001100)$	$f(011100)$	$f(111100)$	$f(101100)$
10	$f(001000)$	$f(011000)$	$f(111000)$	$f(101000)$

$x_4x_5 = 00$

x_5x_4 \ x_3x_2	00	01	11	10
00	$f(000001)$	$f(010001)$	$f(110001)$	$f(100001)$
01	$f(000101)$	$f(010101)$	$f(110101)$	$f(100101)$
11	$f(001101)$	$f(011101)$	$f(111101)$	$f(101101)$
10	$f(001001)$	$f(011001)$	$f(111001)$	$f(101001)$

$x_4x_5 = 01$

x_5x_4 \ x_3x_2	00	01	11	10
00	$f(000010)$	$f(010010)$	$f(110010)$	$f(100010)$
01	$f(000110)$	$f(010110)$	$f(110110)$	$f(100110)$
11	$f(001110)$	$f(011110)$	$f(111110)$	$f(101110)$
10	$f(001010)$	$f(011010)$	$f(111010)$	$f(101010)$

$x_4x_5 = 10$

x_5x_4 \ x_3x_2	00	01	11	10
00	$f(000011)$	$f(010011)$	$f(110011)$	$f(100011)$
01	$f(000111)$	$f(010111)$	$f(110111)$	$f(100111)$
11	$f(001111)$	$f(011111)$	$f(111111)$	$f(101111)$
10	$f(001011)$	$f(011011)$	$f(111011)$	$f(101011)$

$x_4x_5 = 11$

Vediamo ora un esempio :

k	x_1	x_2	x_3	$f(k)$
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

x_3x_2 \ x_1x_0	00	01	11	10
0	1	0	0	1
1	0	0	1	1

Per costruire gli insiemi di copertura (maxtermini =0 e mintermini=1) si raggruppa a seconda della rappresentazione in gruppi di potenze di 2 (2,4,8,16, ...) , avendo così due possibili rappresentazioni : SOP o POS . Da notare che il raggruppamento dei termini non è univoco. Andiamo a vedere degli esempi a 4,5,6 variabili :

x_3x_2 \ x_1x_0	00	01	11	10
00	1	1	1	0
01	1	1	1	0
11	1	0	1	1
10	1	0	1	1

Si semplificano x_1 e x_3

Si semplificano x_0 e x_3

Si semplificano x_2 e x_3

x_4x_3 \ x_2x_1	00	01	11	10
00	1	1	0	1
01	0	1	0	0
11	0	0	0	0
10	1	0	0	1

$x_4 = 0$

x_4x_3 \ x_2x_1	00	01	11	10
00	1	1	0	1
01	0	1	0	0
11	0	0	1	0
10	1	0	0	1

$x_4 = 1$

Da notare che qui il mintermine in verde deriva dalla sovrapposizione delle tabelle .

$ab \backslash cd$	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	1	-	0
10	1	0	0	1

$ef = 00$

$ab \backslash cd$	00	01	11	10
00	0	1	1	0
01	1	0	0	1
11	1	0	0	1
10	0	0	0	0

$ef = 01$

$ab \backslash cd$	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	-	-	0
10	1	0	0	1

$ef = 10$

$ab \backslash cd$	00	01	11	10
00	0	1	1	0
01	1	0	0	1
11	1	0	0	1
10	0	0	0	0

$ef = 11$

Idem per quanto la sovrapposizione , inoltre da notare che l'insieme in blu ha due trattini , ovvero don't care conditions : configurazioni nelle quali la funzione non si potrà mai trovare , quindi a seconda del caso vengono considerati mintermini o maxtermini .

Tornando agli operatori universali : sono necessari tutti e tre ? AND , OR , NOT?? assolutamente no, visto che grazie a De Morgan ed una doppia negazione uno si leva ! :

Esempio: $f(x, y, z) = x + yz + \bar{x}\bar{z} = \bar{\bar{f}}(x, y, z) = \bar{\bar{x} \cdot \bar{y}\bar{z} \cdot \bar{y}\bar{z}}$

Riducendo ulteriormente : usiamo una sola porta logica per esprimere tutta l'algebra booleana : la porta NAND :

- $a|a = \bar{a}$
- $(a|a)|(b|b) = \bar{a} \cdot \bar{b} = a + b$
- $(a|b)|(a|b) = \overline{(a \cdot b)} \cdot \overline{(a \cdot b)} = (a \cdot b) + (a \cdot b) = a \cdot b$
- $a|(a|a) = \bar{a} \cdot \bar{\bar{a}} = a + \bar{a} = 1$
- $(a|(a|a))|(a|(a|a)) = 1|1 = 0$

Dualmente alla NAND , vi è la NOR :

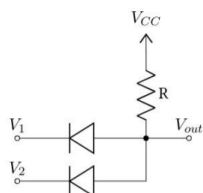
- $a \downarrow a = \bar{a}$
- $(a \downarrow a) \downarrow (b \downarrow b) = \overline{\bar{a} + \bar{b}} = a \cdot b$
- $(a \downarrow b) \downarrow (a \downarrow b) = \overline{(a + b)} = a + b$
- $a \downarrow (a \downarrow a) = 0$
- $(a \downarrow (a \downarrow a)) \downarrow (a \downarrow (a \downarrow a)) = 1$

Di solito si usano questi due operatori per via del numero in costo di componenti, rispetto alle porte non negate (OR / AND) .

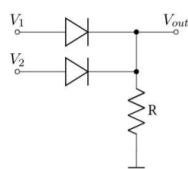
Quindi si parla di reti che accettano input variabile booleane e restituiscono variabili booleane , le quali reti vengono sintetizzate dalle solite regole dell'algebra booleana : in genere si usano delle porte logiche per fare questa sintesi :



Vediamo un pò di storia : queste porte inizialmente furono create ed implementate mediante diodi : in base alla configurazione dei diodi e della resistenza , si hanno due implementazioni duali : AND (primo schema) e OR (secondo schema):



V_{out}	V_1	V_2
V_L	V_L	V_L
V_L	V_L	V_H
V_L	V_H	V_L
V_H	V_H	V_H

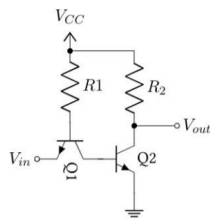


V_{out}	V_1	V_2
V_L	V_L	V_L
V_H	V_L	V_H
V_H	V_H	V_L
V_H	V_H	V_H

Queste porte con questa implementazione avevano un problema : visto che per far funzionare il diodo fa diminuire la tensione applicata al componente , si ha che per più componenti connessi in cascata , si arriva ad una grande attenuazione del segnale :

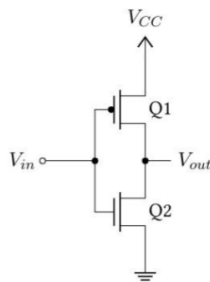
$$V_{out} = \max(V_1, V_2) - V_d$$

Vista questo consumo di energia , a partire dagli anni 60 si è optato per usare una tecnologia a transistor : alcuni servono per commutare il segnale, mentre altri per amplificarlo :

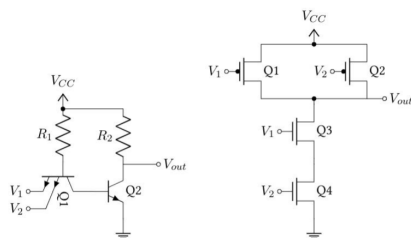


Questa è una porta not!

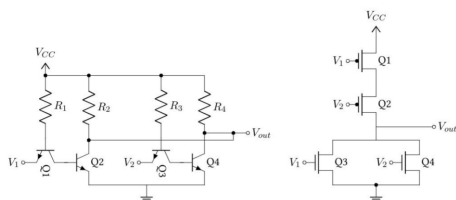
Ma dagli anni 80 si è fatto un miglioramento di questa tecnologia , usando area del dispositivo più piccola e basato su due concetti fondamentali : **rete di pull-up** (collegata direttamente a +Vcc) e **rete di pull-down** (collegata direttamente a massa) :



Inverter



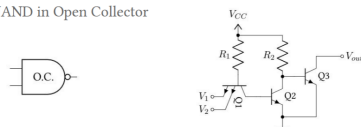
Nand



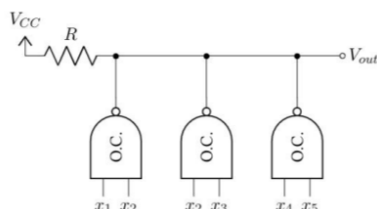
Nor

Vediamo ora ultima tecnologia : **Open Collector** , ovvero tecnologia che aggiunge un transistor il cui collettore è collegato all'uscita della porta : da notare che in questa configurazione si parla di logica negata : la resistenza di pull-up fa salire la tensione quando il circuito è chiuso :

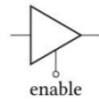
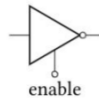
Esempio: NAND in Open Collector



Riassumendo la tecnologia in open collector : si usa quando si ha la necessità di collegare più porte in cascata che possono fornire correnti molto alte, il che potrebbe portare ad un sovraccarico di corrente nel dispositivo (può bruciarsi) . Quindi la corrente che fluisce con questa tecnologia non è direttamente proporzionale al numero di porte usate :



Un primo esempio di circuito combinatorio è il buffer tri-state : fa da ponte tra le diverse parti del circuito , ed è caratterizzato da 3 canali : ingresso, uscita ed enable: In generale l'uscita è uguale ad ingresso se il controllo è attivo , altrimenti stato di alta impedenza (non passa corrente):



IN	ENABLE	OUT
X	0	HiZ
1	1	1
0	1	0